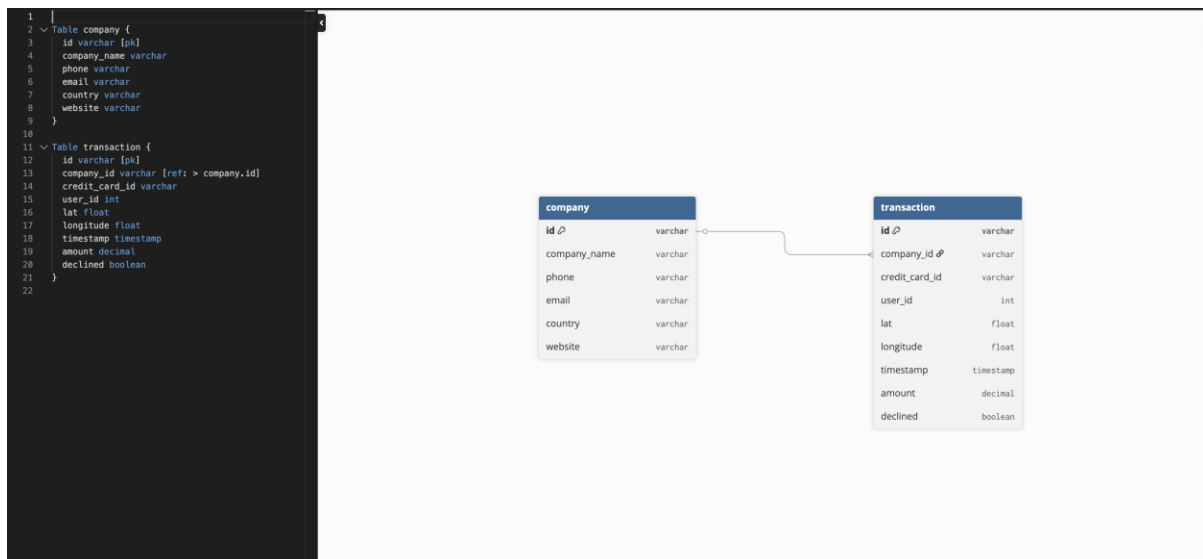


SPRINT 2 – DATA ANALYTICS 2026

LEONARDO ANTONIO RUGGIERO

Ejercicio 1

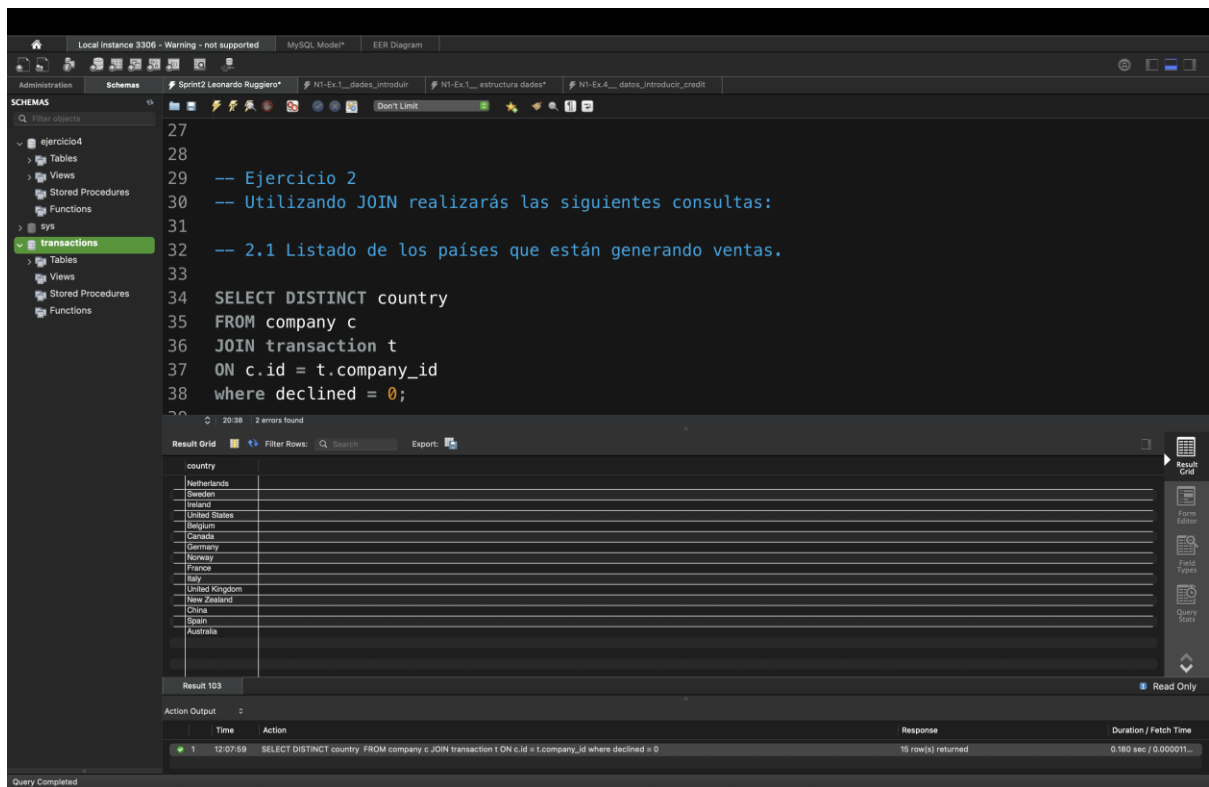
-1.1: A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.



Respuesta:

Esta relación es de **1:N** (uno a muchos), lo que significa que mientras cada empresa aparece una sola vez en su tabla de dimensiones, puede aparecer repetidas veces en la tabla de hechos porque una misma empresa puede generar múltiples transacciones. Para visualizar esta arquitectura, utilicé la herramienta **dbdiagram.io**. Inserté el código de mis tablas en esta plataforma para generar el diagrama relacional de forma limpia y profesional, lo que me permitió confirmar que las conexiones entre hechos y dimensiones fueran correctas.

Ejercicio 2.1



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, containing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The 'transactions' table is highlighted under 'Tables'. The main editor shows a SQL query:

```
27
28
29 -- Ejercicio 2
30 -- Utilizando JOIN realizarás las siguientes consultas:
31
32 -- 2.1 Listado de los países que están generando ventas.
33
34 SELECT DISTINCT country
35 FROM company c
36 JOIN transaction t
37 ON c.id = t.company_id
38 where declined = 0;
```

The 'Result Grid' at the bottom shows the results of the query, displaying a list of countries:

country
Netherlands
Sweden
Ireland
United States
Belgium
Colombia
Germany
Norway
France
Italy
United Kingdom
New Zealand
China
Spain
Australia

The 'Action Output' pane at the bottom shows the execution details:

Time	Action	Response	Duration / Fetch Time
12:07:59	SELECT DISTINCT country FROM company c JOIN transaction t ON c.id = t.company_id where declined = 0	16 row(s) returned	0.180 sec / 0.000011...

Query Completed

Respuesta:

Las relaciones del “JOIN” siempre serán entre el campo “ID” de la tabla “company” y el campo “Company_ID” de la tabla “transaction”. Una vez que hemos creado esa relación entre las dos tablas, en este caso lo que hemos hecho es seleccionar el campo de country de la tabla “company” con un “distinct”, porque no queremos que ninguno de los valores que tenemos nos salgan repetidos. Por fin he añadido “WHERE declined = 0” para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 2.2

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, and 'transactions' is highlighted under the 'Tables' folder. The main editor window contains the following SQL query:

```
40
41 -- 2.2 Desde cuántos países se generan las ventas.
42
43 • SELECT COUNT(DISTINCT country)
44   FROM company c
45   JOIN transaction t
46   ON c.id = t.company_id
47   WHERE declined = 0;
48
49
50
51
52
53
```

Below the query editor, the 'Result Grid' shows the query results:

COUNT(DISTINCT countr...
15

The 'Action Output' pane at the bottom shows the execution details:

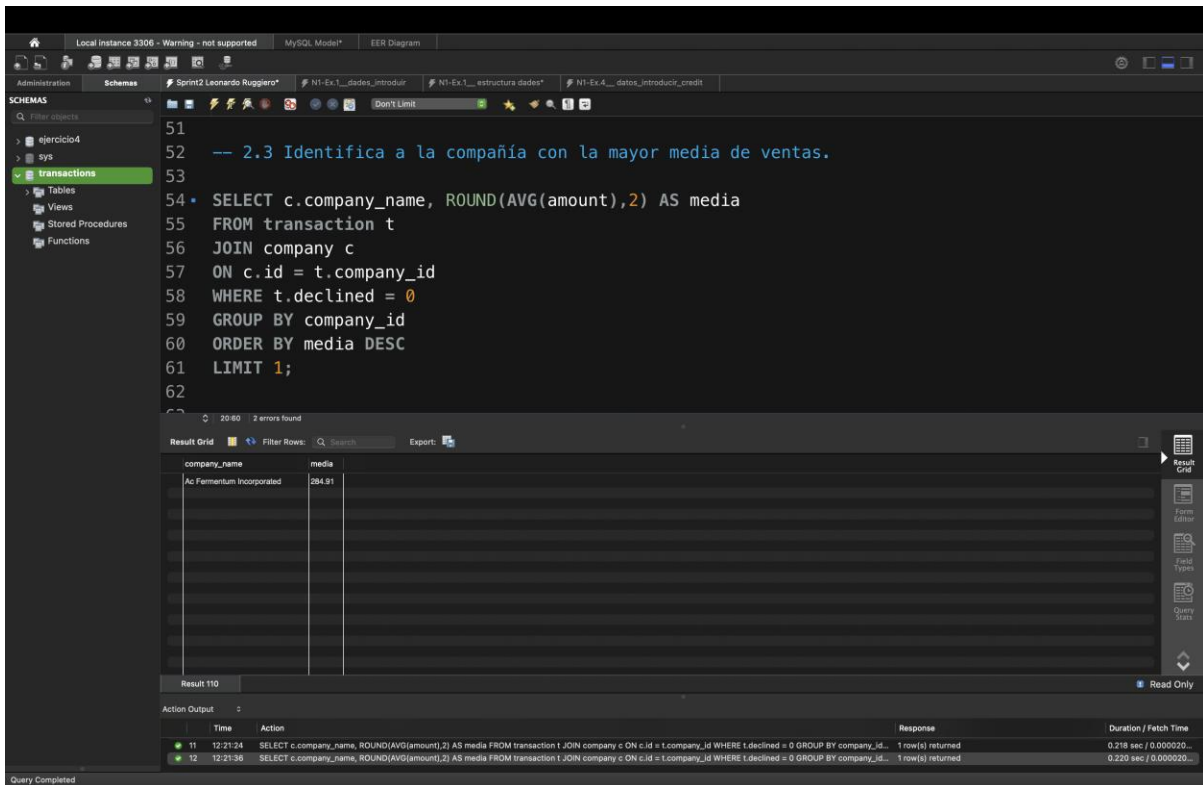
Time	Action	Response	Duration / Fetch Time
20:16:21	SELECT COUNT(DISTINCT country) FROM company c JOIN transaction t ON c.id = t.company_id WHERE declined = 0	1 row(s) returned	0.180 sec / 0.000017...

The status bar at the bottom indicates 'Query Completed'.

Respuesta:

Utilizo la función de agregación “**COUNT(DISTINCT country)**” para obtener el número total de países únicos que han registrado ventas. El uso de **DISTINCT** dentro del conteo es indispensable para asegurar que cada país se contabilice una sola vez, independientemente del volumen de transacciones que tenga asociado. Por fin he añadido “**WHERE declined = 0**” para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 2.3



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
51
52 -- 2.3 Identifica a la compañía con la mayor media de ventas.
53
54 * SELECT c.company_name, ROUND(AVG(amount),2) AS media
55 FROM transaction t
56 JOIN company c
57 ON c.id = t.company_id
58 WHERE t.declined = 0
59 GROUP BY company_id
60 ORDER BY media DESC
61 LIMIT 1;
62
```

The Results tab shows the following data:

company_name	media
Ac Fermentum Incorporated	284.91

The bottom status bar indicates the query was completed successfully.

Respuesta:

Utilizo la función **“AVG”** junto con **“GROUP BY”** para calcular el promedio de ventas por empresa. Aplico **“ORDER BY DESC”** y **“LIMIT 1”** para aislar únicamente el registro con el valor más alto, cumpliendo con el objetivo de identificar la compañía con la mayor media. Además, empleo **ROUND(..., 2)** para cumplir con la normativa de formato para valores monetarios. Por fin he añadido **“WHERE declined = 0”** para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 3.1

The screenshot shows a MySQL IDE interface with a SQL query editor and a results grid. The query is as follows:

```
32
33 -- Utilizando sólo subconsultas (sin utilizar JOIN):
34
35 -- 3.1 Muestra todas las transacciones realizadas por empresas de Alemania.
36
37 * SELECT *
38 FROM transaction t
39 WHERE EXISTS (SELECT company_id FROM company c WHERE c.id = t.company_id AND c.country = 'Germany' AND t.declined = 0);
40
```

The results grid displays the following columns: id, credit_card..., company_id, user_id, lat, longitude, timestamp, amount, and declined. The results show a list of transactions for companies in Germany, with columns truncated for brevity.

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
00138C3B-20D-4C03-9487-63A267E9B964	C6S-4889	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0018C186-38A-420C-8154-E2B8F80CA8B9	C6S-5010	b-2222	480	41.3814	12.1876	2020-12-17 18:12:37	218.90	0
00201A11-2E62-44C4-941D-196F-C8D877F9	C6U-3512	b-2222	185	55.5704	3.68129	2021-01-22 23:44:27	433.04	0
00235618-0A3C-4D49-90C8-63A842D06923	C6S-8137	b-2222	3058	58.8421	18.729	2020-09-09 15:43:19	263.14	0
00245419-31A-485C-9815-162A7602C28	C6S-8988	b-2222	4417	51.1581	8.68905	2024-02-15 09:10:11	142.0	0
00867139-4B52-4FFA-8E73-820378F04A84	C6S-4870	b-2222	289	51.1866	10.4869	2019-03-09 19:37:49	524.84	0
0074F4D0-3D71-4827-8758-0498314623A	C6S-4061	b-2222	3500	56.7016	4.56325	2016-12-26 23:06:57	491.90	0
00A8B0C0-36C8-42C8-B410-138E72C05AAB	C6S-4787	b-2222	2218	55.7852	3.78245	2021-04-29 03:56:59	167.15	0
00E0804-8920-4708-ABE5-3252268626D	C6S-4683	b-2222	402	58.708	8.12993	2019-02-27 15:25:16	141.66	0
00DA0383-E548-4377-8ED1-3C5C256FF2F	C6S-9223	b-2222	4642	51.1742	10.2627	2019-03-21 11:47:34	325.62	0
00D511D5-E0D1-48AD-33A3-17AD-1E8A50F	C6S-7661	b-2222	3109	45.7640	4.83109	2024-07-28 18:20:49	242.53	0
01449C60-98E9-4DE3-9810-798C8A6056F	C6S-5424	b-2222	643	47.0163	2.26054	2024-08-17 19:37:14	451.71	0
0176EAC7-241E-42DA-A889-9F246DBF4D3F	C6S-7510	b-2222	2629	52.0019	4.29484	2021-08-28 16:20:38	9.46	0
01A8DA0B-06E2-42A0-A131-AEE6FF11B789	C6S-5583	b-2222	416	51.7438	5.17479	2020-01-28 01:15:07	368.43	0
01F1C7ED-0823-442D-AE5E-3134D5004866	C6S-6776	b-2222	2195	58.6697	18.6697	2022-12-17 09:40:14	168.79	0
01FAB051-1B08-441B-96FC-428A3F06097	C6S-5831	b-2222	965	55.3405	-3.3863	2020-04-30 08:57:27	333.48	0
0203714C-5D01-4502-A9E6-2865874F61B	C6S-5103	b-2222	292	45.5352	2.99198	2019-04-14 15:52:13	155.59	0
0248B812-89F5-4B21-B8E9-E0D51800A579	C6S-9328	b-2222	4747	42.0816	12.8326	2016-07-01 01:33:41	268.23	0
02DF683C-E822-42D3-B78E-15A0A3A27694	C6S-4869	b-2222	276	60.8186	18.5276	2021-02-03 14:12:33	127.23	0
0310290E-F9A2-4448-95EE-87000086534D	C6U-5760	b-2222	159	55.355	3.44611	2020-04-05 19:29:29	68.38	0

The results grid shows 79 rows of data. The bottom of the screen displays the query execution details: "Query Completed", "Time: 12:35:00", "Action: SELECT * FROM transaction t WHERE EXISTS (SELECT company_id FROM company c WHERE c.id = t.company_id AND c.country = 'Germany' AND t.declined = 0)", "Response: 13289 rows returned", and "Duration / Fetch Time: 0.0017 sec / 0.066 sec".

Respuesta:

Siguiendo el requerimiento del enunciado de no utilizar JOIN, empleo una **subconsulta** sobre la tabla company para recuperar los identificadores (id) asociados a Alemania. Utilizo este conjunto de datos como filtro en la cláusula “**WHERE EXISTS**” de la consulta principal sobre transaction. Adicionalmente, he incluido la condición “declined = 0” para asegurar que el listado solo muestre transacciones efectivas, excluyendo aquellas que fueron declinadas, optimizando así la relevancia de los datos obtenidos.

Ejercicio 3.2

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, showing its tables: 'company', 'credit_card', 'transaction', and 'user'. The main editor window contains the following SQL query:

```
74
75 -- 3.2 Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones
76
77 SELECT c.company_name
78 FROM company as c
79 WHERE EXISTS (
80     SELECT id
81     FROM transaction as t
82     WHERE t.company_id = c.id AND t.amount > (SELECT AVG(amount) FROM transaction) AND t.declined = 0);
83
84
85
86
87
```

Below the query editor, the 'Result Grid' shows the results of the query. The first column is 'company_name' and the second column is 'company_id'. The results are:

company_name	company_id
Ac Fomentum Incorporated	
Magna A Neque Industries	
Fusce Corp	
Consectetur Inc Incorporated	
Arto laculis Nec Foundation	
Donec Ltd	
Ipsum Nunc Ltd	

The 'Action Output' pane at the bottom shows the execution details:

Time	Action	Response	Duration / Fetch Time
2019-21	SELECT c.company_name FROM company as c WHERE EXISTS (SELECT id FROM transaction as t WHERE t.company_id = c.id AND t.amount > (SELECT AVG(amo... 100 row(s) returned		0.077 sec / 0.000022...

Query Completed

Respuesta:

Para resolver este ejercicio sin utilizar “JOIN”, empleo la cláusula “**WHERE EXISTS**” con una **subconsulta correlacionada**. Esta estructura permite filtrar las empresas (company) basándose en la existencia de al menos una transacción en la tabla transaction que cumpla con los criterios especificados. Dentro de la subconsulta, comparo el importe de cada transacción (t.amount) contra una **subconsulta escalar** que calcula el promedio global (AVG(amount)) de toda la tabla. He decidido mantener la condición t.declined = 0 para asegurar que el cálculo y el filtrado se realicen exclusivamente sobre transacciones exitosas, garantizando la integridad de la lógica de negocio solicitada.

Ejercicio 3.3

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, containing tables like 'company', 'credit_card', 'transaction', and 'user'. The main editor shows a SQL query:

```
86
87
88
89 -- 3.3 Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas
90 -- empresas.
91
92 * SELECT *
93 FROM company c
94 WHERE NOT EXISTS ( SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1);
95
96
97
98
99
```

Below the query, the 'Result Grid' shows 156 rows of data with columns: id, company_name, phone, email, country, and website. The first few rows are:

id	company_name	phone	email	country	website
1	Mus Aenean Eget Foundation	06 25 15 52 43	mi.duis@hotmail.net	Sweden	https://instagram.com/group9
2	Dis Parturient Institute	05 36 29 19 74	janet@protonmail.org	Ireland	https://google.com/one
3	Eit Exam Limited Associates	07 69 75 17 45	dfrose@google.co.uk	Canada	https://yahoo.com/
4	Nunc Intendum Incorporated	05 18 15 48 13	hqn@outlook.com	Germany	https://wikipedia.org/en-us
5	A Institute	03 34 91 68 65	melus.alquari@google.edu	Belgium	https://reddit.com/
6	Integer Molis Corp.	03 12 50 45 54	ita.eros@protonmail.ca	Italy	https://mellit.com/group9
7	Amet Institute	06 33 40 21 33	hulam.loboris.quam@outlook.net	Australia	https://nytimes.com/one

The 'Action Output' pane at the bottom shows two successful queries:

	Time	Action	Response	Duration / Fetch Time
1	20:24:34	SELECT * FROM company c WHERE NOT EXISTS (SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1)	14 row(s) returned	0.034 sec / 0.000014...
2	20:26:27	SELECT * FROM company c WHERE NOT EXISTS (SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1)	14 row(s) returned	0.034 sec / 0.000017...

Respuesta:

Utilize la cláusula **EXISTS** con una **subconsulta correlacionada** para filtrar la tabla company. He vinculado ambas tablas mediante la condición `t.company_id = c.id` dentro de la subconsulta para verificar qué empresas tienen registros asociados en la tabla de transacciones. He aplicado el filtro `declined = 1` para limitar el resultado únicamente a aquellas empresas que presentan transacciones declinadas.

Ejercicio 4

Local instance 3306 - Warning - not supported MySQL Mode*

Administration Schemas Sprint2 Leonardo Ruggiero*

SCHEMAS

Filter objects

sys

transactions

Tables

company

transaction

user

Views

Stored Procedures

Functions

55 -- Ejercicio 4

56

57 -- Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito.

58 -- La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas

59 -- ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento

60 -- denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

61

62

63 CREATE TABLE credit_card (

64 id VARCHAR(15) PRIMARY KEY,

65 iban VARCHAR(50),

66 pan VARCHAR(20),

67 pin VARCHAR(4),

68 cvv VARCHAR(3),

69 expiring_date VARCHAR(10)

70);

71

72

73

74

75

76

77

78

79

80

0.1

1-B1 2 errors found

Result Grid

Filter Rows: Search Edit Export/Import

Id credit_card... company_id user_id lat longitude timestamp amount declined

transaction 143

Action Output

Time Action Response Duration / Fetch Time

52... 13:12:05 CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(4), cvv VARCHAR(3), expiring_date VARCHAR(10)) 0 row(s) affected 0.0079 sec

Query Completed

Local instance 3306 - Warning - not supported MySQL Mode* EER Diagram

Administration Schemas Sprint2 Leonardo Ruggiero* NI-Ex-1_datos_introducir NI-Ex-1_estructura datos NI-Ex-1_datos_introducir_credit

SCHEMAS

Filter objects

ejercicio4

Tables

Views

Stored Procedures

Functions

sys

transactions

Tables

company

credit_card

transaction

user

Views

Stored Procedures

Functions

1 -- Insertamos datos de credit_card

2 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2938', 'TR30195031221357681636661', '5424465566813633', '3257', '984', '10/30/22');

3 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2945', 'D026854763748537475216568689', '5142423821948828', '9800', '887', '08/24/23');

4 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2952', 'B6451V0L52718525680255', '4556 453 55 5287', '4598', '438', '06/29/21');

5 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2959', '07242477244335841535', '37246137749375', '3583', '667', '02/24/23');

6 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2966', 'B072LX015627628377333', '448846 886747 7265', '4900', '130', '10/29/24');

7 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2973', 'PT87866228135802429456346', '544 5854 5434 384', '8760', '887', '01/30/25');

8 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2980', 'D63924188183886277136', '482408 7145845969', '5875', '596', '07/24/22');

9 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2987', 'G689681434837748781813', '3763 747687 7666', '2290', '797', '10/31/23');

10 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2994', 'B8657214428368866765294', '344283273252593', '7545', '595', '02/28/22');

11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3001', 'CY49807426654774581266832110', '511722 924833 2244', '9562', '867', '09/16/22');

12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3008', 'LUS07216693616119238', '4485744464433884', '1856', '740', '04/05/25');

13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822', '01/31/22');

14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3022', 'GT9169516285856977423121857', '5164 1379 4842 3951', '5610', '342', '04/25/25');

15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3029', 'AZ62317143982441418123739746', '3429 279566 77631', '9708', '589', '09/02/23');

16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3036', 'AZ3933682925842865843941994', '3768 451556 48766', '2232', '565', '10/27/25');

17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3043', 'T8648814318514852179535', '455876 6457483635', '5969', '198', '06/07/25');

18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3050', 'PS167744369175838331854477', '4824087122722', '4834', '126', '10/09/23');

19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3057', 'U0931822574697545315', '3484 621767 21237', '6805', '848', '09/14/25');

20 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3064', 'PS14696554544925377627273133', '3467 732741 26818', '3865', '408', '06/03/25');

21 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3071', 'N08023814763512', '3464 789562 23352', '6625', '661', '12/20/23');

22 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3078', 'TS825127145884623279548733', '4539 322 74 2377', '9405', '720', '03/08/23');

23 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3085', 'B863114723972437', '5266 3346 1135 1687', '7241', '413', '05/10/23');

24 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3092', 'R065L5001166122125447487', '3488 754223 46253', '9417', '594', '12/19/22');

25 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3099', 'PT26185273556823785537218', '448 55418 98863 789', '5612', '564', '01/22/23');

26 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3106', 'AT6845163775136592', '349547146395283', '9733', '289', '01/27/24');

27 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3113', 'IE26L6747732173572752', '34184822877471', '9011', '287', '06/12/21');

28 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3120', 'RS7265576666166237144', '527646 53375 6577', '7658', '265', '01/16/21');

29 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3127', 'PT8353461438644342816864', '4716 443 46 4368', '8038', '924', '01/16/23');

30 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3134', 'B6239Y052668951824779', '5146 3453 9766 2168', '7260', '935', '08/24/25');

100%

Action Output

Time Action Response Duration / Fetch Time

49... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9577', 'XX405448186289720388404508', '829682284225053', '1985', '220', '07/27/29') 1 row(s) affected 0.00016 sec

49... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9578', 'XX965338931053088901924906', '9099040779637171', '2277', '224', '01/25/28') 1 row(s) affected 0.00014 sec

49... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9574', 'XX86276758361432688520775', '3717833138656', '5084', '708', '11/28/27') 1 row(s) affected 0.00029 sec

49... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9575', 'XX499881260726267196473', '8086035027377786', '7266', '960', '10/25/29') 1 row(s) affected 0.00015 sec

49... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9576', 'XX32970260577264772007', '4554223878064007', '4884', '724', '06/28/29') 1 row(s) affected 0.00016 sec

50... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9577', 'XX15891407898408633147086', '81043727858107', '5864', '772', '10/29/26') 1 row(s) affected 0.00016 sec

50... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9578', 'XX91953964646110567870254', '899808823081411', '2872', '772', '07/28/27') 1 row(s) affected 0.00015 sec

50... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9579', 'XX29639309158770202131236', '9690006048678689', '8378', '134', '12/25/25') 1 row(s) affected 0.00014 sec

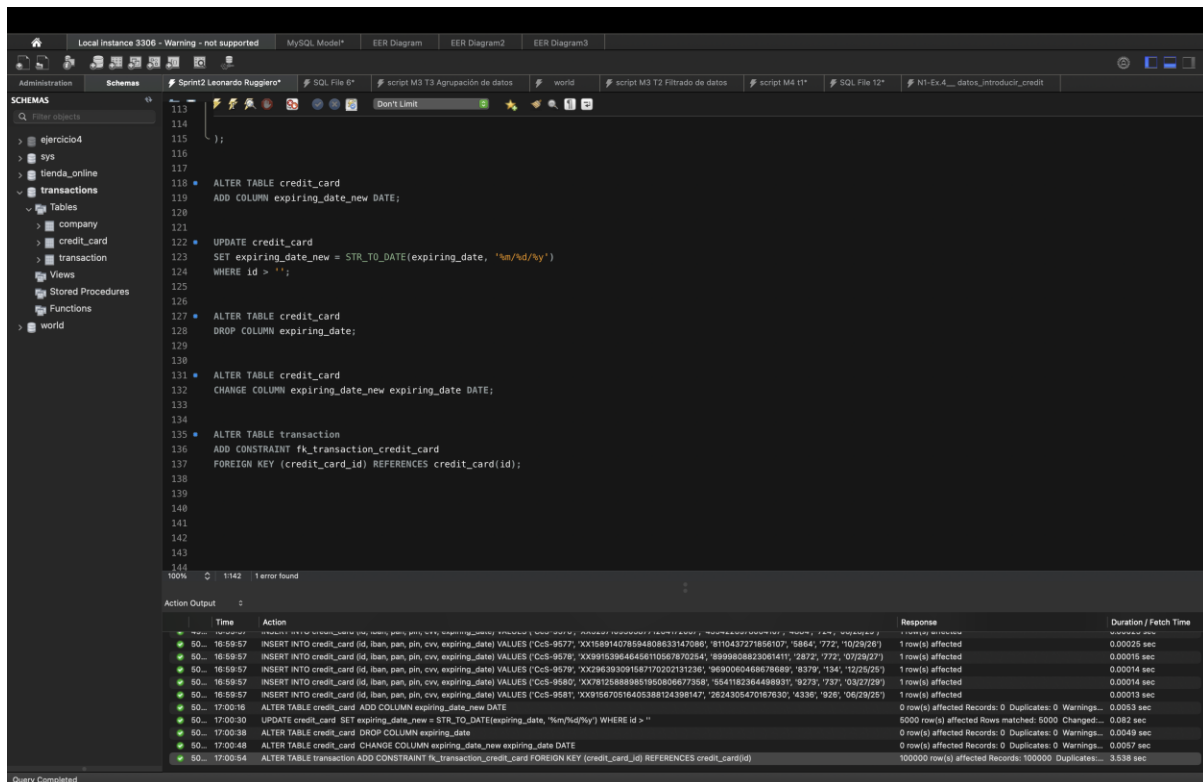
50... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9580', 'XX78126888981950806677368', '944182384488931', '3273', '737', '03/27/29') 1 row(s) affected 0.00015 sec

50... 19:40:14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9581', 'XX8158705164053812439847', '5824385470167839', '4336', '926', '06/28/25') 1 row(s) affected 0.00016 sec

Query Completed

RESPUESTA:

Para resolver este ejercicio, primero creé la tabla `credit_card` definiendo la columna `expiring_date` temporalmente como `VARCHAR`. Hice isto porque los datos del archivo original venían en formato de texto ('MM/DD/AA') e insertarlos directamente como fecha iba a dar error.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected. The main editor shows a series of SQL queries for creating and modifying the 'credit_card' table. The 'Action Output' pane at the bottom shows the execution results of these queries.

```
113
114
115
116
117
118 ALTER TABLE credit_card
119 ADD COLUMN expiring_date_new DATE;
120
121
122 UPDATE credit_card
123 SET expiring_date_new = STR_TO_DATE(expiring_date, '%m/%d/%y')
124 WHERE id > '1';
125
126
127 ALTER TABLE credit_card
128 DROP COLUMN expiring_date;
129
130
131 ALTER TABLE credit_card
132 CHANGE COLUMN expiring_date_new expiring_date DATE;
133
134
135 ALTER TABLE transaction
136 ADD CONSTRAINT fk_transaction_credit_card
137 FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);
138
139
140
141
142
143
144
```

Time	Action	Response	Duration / Fetch Time
50... 16:59:57	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C05-9577', 'XX158914078594808633147086', '8110437271856107', '5864', '772', '10/28/26')	1 row(s) affected	0.00025 sec
50... 16:59:57	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C05-9578', 'XX991039646456110567870254', '89980882306141', '2872', '772', '07/28/27')	1 row(s) affected	0.00015 sec
50... 16:59:57	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C05-9579', 'XX296383091659170202191236', '9690060486876689', '8379', '184', '10/28/25')	1 row(s) affected	0.00014 sec
50... 16:59:57	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C05-9580', 'XX791258889881950806677358', '5541182364488931', '9272', '727', '03/27/28')	1 row(s) affected	0.00014 sec
50... 16:59:57	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('C05-9581', 'XX915670516405388124398147', '2624305470167630', '4336', '926', '06/28/25')	1 row(s) affected	0.00013 sec
50... 17:00:16	ALTER TABLE credit_card ADD COLUMN expiring_date_new DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.0053 sec
50... 17:00:30	UPDATE credit_card SET expiring_date_new = STR_TO_DATE(expiring_date, '%m/%d/%y') WHERE id > '1'	5000 row(s) affected Rows matched: 5000 Changed: 5000	0.082 sec
50... 17:00:38	ALTER TABLE credit_card DROP COLUMN expiring_date	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.0549 sec
50... 17:00:48	ALTER TABLE credit_card CHANGE COLUMN expiring_date_new expiring_date DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.0057 sec
50... 17:00:54	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_card FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.538 sec

Query Completed

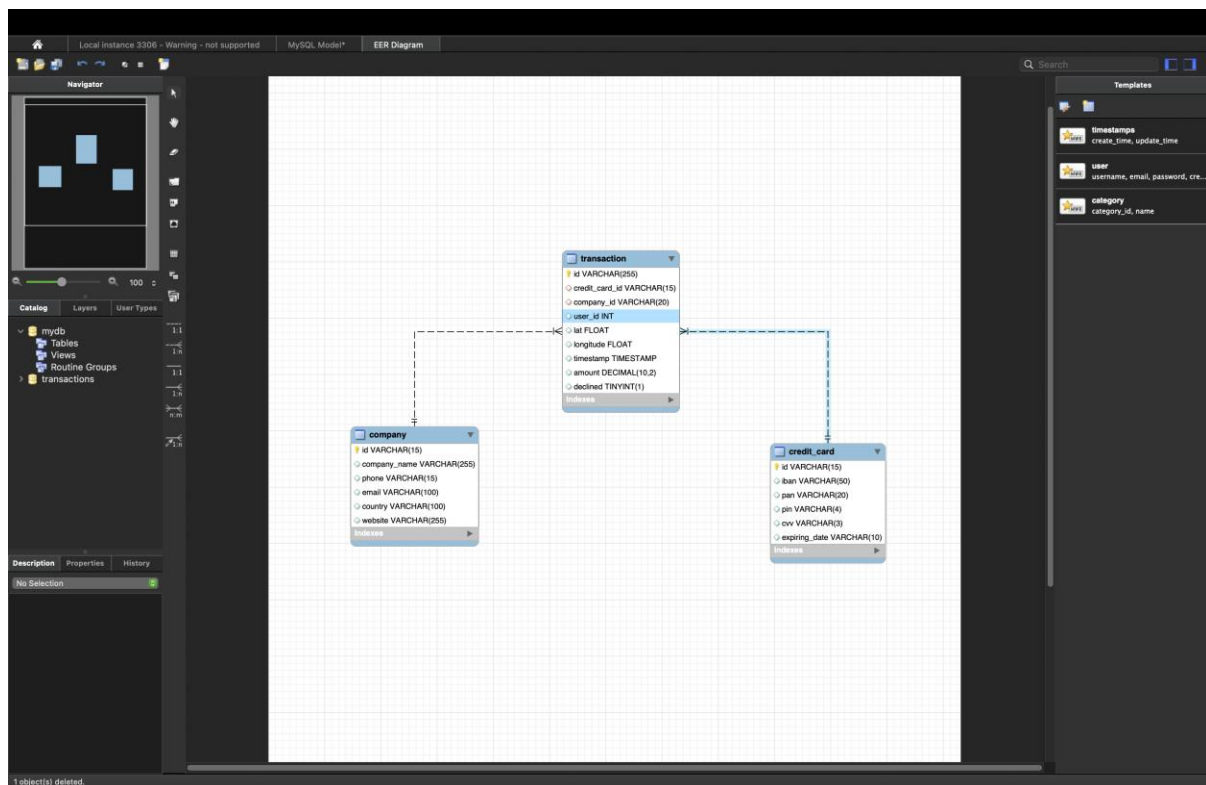
RESPUESTA:

Una vez que subí todos los registros, utilicé la función `STR_TO_DATE` para transformar esos textos en fechas reales. Sin embargo, para aplicarlo a toda la tabla me encontré con la típica traba de seguridad del MySQL Workbench (`SQL_SAFE_UPDATES`), que no te deja actualizar muchas filas a la vez. Para no tener que usar comandos `SET` que desactivaran esa protección por completo, apliqué un truco en la cláusula `WHERE`: filtré usando la Clave Primaria con la condición `WHERE id > '1'`. Como el `id` es un índice, el sistema validó la operación como segura y me dejó procesar todos los registros de una sola vez de forma nativa.

Con los datos ya convertidos en una columna nueva de tipo `DATE` (`expiring_date_new`), el paso final de la migración fue limpiar la tabla: eliminé a columna antigua de texto con

um DROP COLUMN y renombré la nueva columna con el nombre oficial de expiring_date.

Finalmente, con la estructura de los cartões ya corregida y en el formato correcto, vinculé formalmente la tabla transaction con credit_card añadiendo la restricción de Clave Foránea (FOREIGN KEY) directamente, dejando el modelo relacional completamente conectado y seguro."



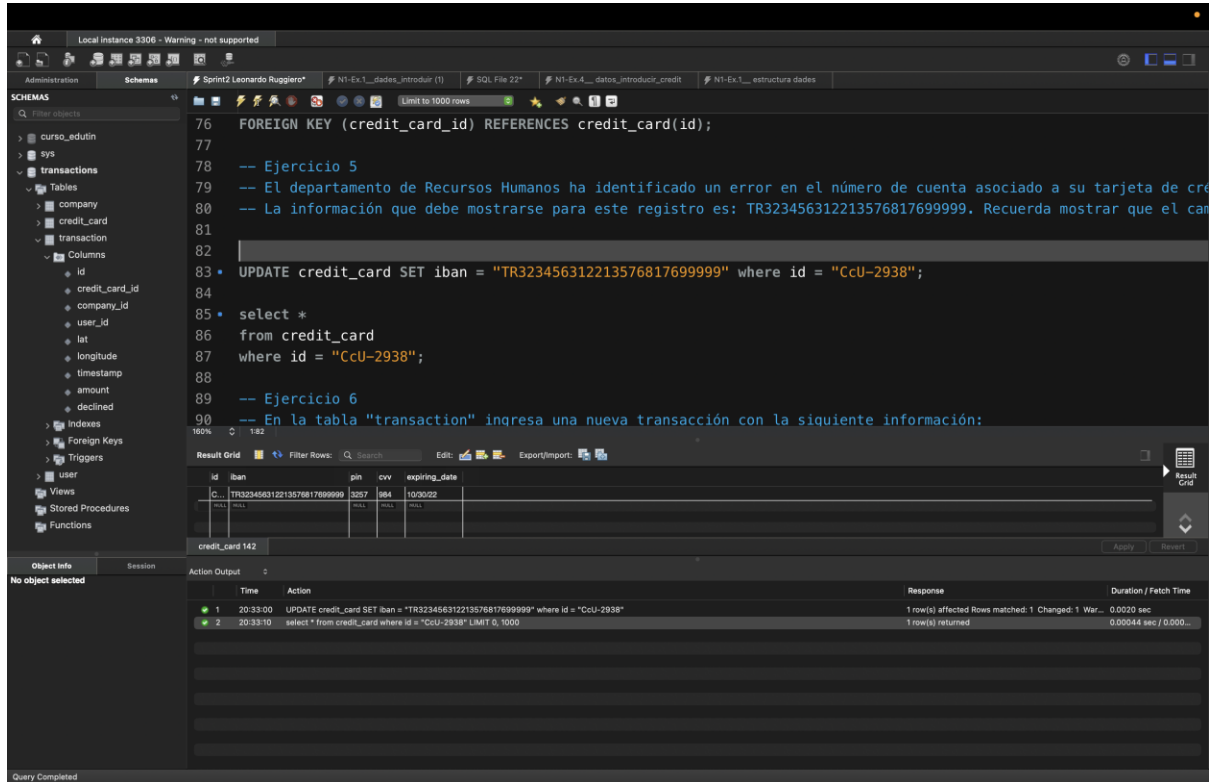
RESPUESTA:

Para visualizar cómo quedaron conectadas todas mis tablas, generé un diagrama de modelo relacional directamente en **MySQL Workbench**. No lo dibujé a mano, sino que utilicé la herramienta automática del programa siguiendo la ruta **Database > Reverse Engineer**.

Lo que hice fue pedirle al sistema que tomara las tablas que ya había creado y las transformara en un esquema visual. Al ejecutar este proceso, el Workbench analizó mis llaves primarias y relaciones para mostrarme el mapa completo de la base de datos. Es un paso que siempre hago para asegurarme de que la arquitectura de la

información sea lógica y que todas las conexiones entre transacciones, productos y usuarios funcionen correctamente.

Ejercicio 5



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
76 FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);
77
78 -- Ejercicio 5
79 -- El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito.
80 -- La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el campo
81
82
83 • UPDATE credit_card SET iban = "TR323456312213576817699999" where id = "CcU-2938";
84
85 • select *
86   from credit_card
87  where id = "CcU-2938";
88
89 -- Ejercicio 6
90 -- En la tabla "transaction" ingresa una nueva transacción con la siguiente información:
```

The results grid shows the following data:

id	iban	pin	cvv	expiring_date
C	TR323456312213576817699999	3337	964	10/30/22

The Action Output pane shows the following actions:

Time	Action	Response	Duration / Fetch Time
20:33:00	UPDATE credit_card SET iban = "TR323456312213576817699999" where id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 War...	0.0020 sec
20:33:10	select * from credit_card where id = "CcU-2938" LIMIT 0, 1000	1 row(s) returned	0.0044 sec / 0.000...

Respuesta

Primero, utilicé el código "select * from credit_card where id = "CcU-2938";" para verificar los valores de la columna data. Luego, cambié el valor del IBAN que antes sería "TR301950312213576817638661" para "TR323456312213576817699999" usando "UPDATE credit_card SET iban = "TR323456312213576817699999" where id = "CcU-2938";" donde UPDATE modifica el valor existente al siguiente valor introducido en la línea de código. El "WHERE id" especifica qué PRIMARY KEY se modificará.

Ejercicio 6

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
155 -- Ejercicio 6
156 -- En la tabla "transaction" ingresa una nueva transacción con la siguiente información:
157
158 • INSERT INTO company(id)
159   VALUES ('b-9999');
160
161 • INSERT INTO credit_card(id)
162   VALUES ('Ccu-9999');
163
164 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
165   VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'Ccu-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
166
167 • SELECT *
168   FROM transaction
169   WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
170
171
```

The Result Grid shows the output of the SELECT query:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	Ccu-9999	b-9999	9999	829.999	-117.999	111.11	0	

The Action Output shows the execution of the queries:

Time	Action	Response	Duration / Fetch Time
17:32:35	SELECT * FROM transaction WHERE user_id = 'b-9999'	0 row(s) returned	0.00051 sec / 0.0000...
17:32:40	INSERT INTO company(id) VALUES ('b-9999')	Error Code: 1062. Duplicate entry 'b-9999' for key 'c...	0.00049 sec
17:33:10	DELETE FROM company where id = 'b-9999'	1 row(s) affected	0.0010 sec
17:33:32	DELETE FROM credit_card where id = 'Ccu-9999'	1 row(s) affected	0.00076 sec
17:33:36	SELECT * FROM transaction WHERE user_id = 'b-9999'	0 row(s) returned	0.00060 sec / 0.000...
17:33:45	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'Ccu-999...	Error Code: 1452. Cannot add or update a child row: a...	0.00060 sec
17:34:09	INSERT INTO company(id) VALUES ('b-9999')	1 row(s) affected	0.00087 sec
17:34:14	INSERT INTO credit_card(id) VALUES ('Ccu-9999')	1 row(s) affected	0.00064 sec
17:34:19	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'Ccu-999...	1 row(s) affected	0.0014 sec
17:34:47	SELECT * FROM transaction WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD'	1 row(s) returned	0.00049 sec / 0.000...

Respuesta:

En este ejercicio, utilicé las funciones "INSERT INTO" y "VALUES" para agregar valores a la nueva columna solicitada. Sin embargo, tuve que crear nuevos campos COMPANY y CREDIT_CARD porque los valores solicitados no existían en la tabla, lo que invalidaba mi función de inserción. Después de crear estos nuevos valores, intenté insertar la nueva columna nuevamente y la inserción fue exitosa.

Ejercicio 7

The screenshot shows a database management tool interface. On the left, a sidebar displays a tree view of database objects, including 'transaction' and 'credit_card'. The main area shows a SQL query editor with the following text:

```
-- Ejercicio 7
-- Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

130 select pan
131 from credit_card;
132
133 ALTER TABLE credit_card DROP column pan
```

Below the query editor, a 'Results Grid' displays the results of the query. The grid has columns: 'id', 'credit_card...', 'company_id', 'user_id', 'lat', 'longitude', 'timestamp', 'amount', and 'declined'. The first row shows 'null' for 'id' and 'credit_card...', and 'null' for the other columns.

At the bottom, an 'Action Output' panel shows the execution of the query. It includes a table with columns: 'Time', 'Action', 'Response', and 'Duration / Fetch Time'.

Time	Action	Response	Duration / Fetch Time
21:44:35	select pan from credit_card LIMIT 0, 1000	1000 row(s) returned	0.0012 sec / 0.00011...
21:44:47	ALTER TABLE credit_card DROP column pan	0 row(s) affected Records: 0 Duplicates: 0 Warnin...	0.0098 sec

The bottom status bar indicates 'Query Completed'.

Respuesta:

Utilicé la función "ALTER TABLE" para modificar la tabla "credit_card" y "DROP COLUMN" para eliminar la columna "pan" de la tabla.

Ejercicio 8

Local Instance 3306 - Warning - not supported MySQL Model* EER Diagram EER Diagram2 EER Diagram3

Administration Schemas

Schemas

- ejercicio8
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
- tienda_online
- transactions
- world

190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222

```
-- transactions.csv  
-- Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:  
-- COLOCAR NOT EXISTS  
CREATE DATABASE IF NOT EXISTS ejercicio8;  
USE ejercicio8;  
CREATE TABLE user (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  surname VARCHAR(100),  
  phone VARCHAR(50),  
  email VARCHAR(150),  
  birth_date VARCHAR(50),  
  country VARCHAR(100),  
  city VARCHAR(100),  
  postal_code VARCHAR(20),  
  address VARCHAR(255)  
);  
CREATE TABLE company (  
  id VARCHAR(15) PRIMARY KEY,  
  company_name VARCHAR(255),  
  phone VARCHAR(50),  
  email VARCHAR(100),  
  country VARCHAR(100),  
  website VARCHAR(255)  
);  
CREATE TABLE credit_card (  
  id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date VARCHAR(100)
```

Action Output

	Time	Action	Response	Duration / Fetch Time
1	18:33:04	CREATE DATABASE IF NOT EXISTS ejercicio8	1 row(s) affected	0.0011 sec
2	18:33:04	USE ejercicio8	0 row(s) affected	0.00018 sec
3	18:33:17	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(50), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), address VARCHAR(255))	0 row(s) affected	0.0053 sec
4	18:33:17	CREATE TABLE company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(100), website VARCHAR(255))	0 row(s) affected	0.0043 sec
5	18:33:17	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date VARCHAR(100))	0 row(s) affected	0.0057 sec
6	18:33:17	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10,2), declined TINYINT(1), product_ids VARCHAR(255), user_id INT, lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (user_id) REFERENCES user(id))	0 row(s) affected, 1 warning(s): 1681 Integer display...	0.015 sec

Query Completed

Local Instance 3306 - Warning - not supported MySQL Model* EER Diagram EER Diagram2 EER Diagram3

Administration Schemas

Schemas

- ejercicio8
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
- tienda_online
- transactions
- world

217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248

```
country VARCHAR(100),  
website VARCHAR(255)  
);  
CREATE TABLE credit_card (  
  id VARCHAR(15) PRIMARY KEY,  
  user_id INT,  
  iban VARCHAR(50),  
  pan VARCHAR(20),  
  pin VARCHAR(10),  
  cvv VARCHAR(10),  
  track1 VARCHAR(255),  
  track2 VARCHAR(255),  
  expiring_date VARCHAR(100)  
);  
CREATE TABLE transaction (  
  id VARCHAR(255) PRIMARY KEY,  
  card_id VARCHAR(15),  
  business_id VARCHAR(15),  
  timestamp TIMESTAMP,  
  amount DECIMAL(10,2),  
  declined TINYINT(1),  
  product_ids VARCHAR(255),  
  user_id INT,  
  lat FLOAT,  
  longitude FLOAT,  
  FOREIGN KEY (card_id) REFERENCES credit_card(id),  
  FOREIGN KEY (business_id) REFERENCES company(id),  
  FOREIGN KEY (user_id) REFERENCES user(id)  
);
```

Action Output

	Time	Action	Response	Duration / Fetch Time
1	18:33:04	CREATE DATABASE IF NOT EXISTS ejercicio8	1 row(s) affected	0.0011 sec
2	18:33:04	USE ejercicio8	0 row(s) affected	0.00018 sec
3	18:33:17	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(50), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), address VARCHAR(255))	0 row(s) affected	0.0053 sec
4	18:33:17	CREATE TABLE company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(100), website VARCHAR(255))	0 row(s) affected	0.0043 sec
5	18:33:17	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date VARCHAR(100))	0 row(s) affected	0.0057 sec
6	18:33:17	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10,2), declined TINYINT(1), product_ids VARCHAR(255), user_id INT, lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (user_id) REFERENCES user(id))	0 row(s) affected, 1 warning(s): 1681 Integer display...	0.015 sec

Query Completed

Local instance 3306 - Warning - not supported

Administration Schemas MySQL Model* EER Diagram EER Diagram2 EER Diagram3

Schemas

- ejercicio8
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
- tienda_online
- transactions
- world

```
business_id VARCHAR(15),
timestamp TIMESTAMPT,
amount DECIMAL(10,2),
declined TINYINT(1),
product_ids VARCHAR(255),
user_id INT,
lat FLOAT,
longitude FLOAT,
FOREIGN KEY (card_id) REFERENCES credit_card(id),
FOREIGN KEY (business_id) REFERENCES company(id),
FOREIGN KEY (user_id) REFERENCES user(id)
};

LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ american_users.csv'
INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ european_users.csv'
INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ companies.csv'
INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ credit_cards.csv'
INTO TABLE credit_card FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
```

LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ transactions.csv'
INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;

100% 3:248 1 error found

Action	Time	Action	Response	Duration / Fetch Time
1	18:33:04	USE ejercicio8	0 row(s) affected	0.00018 sec
2	18:33:17	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(10), country VARCHAR(100))	0 row(s) affected	0.00032 sec
3	18:33:17	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(20), track2 VARCHAR(20))	0 row(s) affected	0.00032 sec
4	18:33:17	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMPT, amount DECIMAL(10,2), declined TINYINT(1), product_ids VARCHAR(255))	0 row(s) affected, 1 warning(s): '1681' Integer display...	0.015 sec
5	18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ american_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' EN...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0	0.013 sec
6	18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ european_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' EN...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0	0.053 sec
7	18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ companies.csv' INTO TABLE company FIELDS TERMINATED BY ';' EN...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0	0.0026 sec
8	18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ';' EN...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0	0.051 sec
9	18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializaco 2026 SQL/N1-Ex.8__ transactions.csv' INTO TABLE transaction FIELDS TERMINATED BY ';' EN...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0	2.872 sec

Query Completed

Local instance 3306 - Warning - not supported

Administration Schemas MySQL Model* EER Diagram EER Diagram2 EER Diagram3

Schemas

- sys
- transactions

```
lat FLOAT,
longitude FLOAT,
FOREIGN KEY (card_id) REFERENCES credit_card(id),
FOREIGN KEY (business_id) REFERENCES company(id),
FOREIGN KEY (user_id) REFERENCES user(id)
};

SET FOREIGN_KEY_CHECKS = 0;

LOAD DATA INFILE '/tmp/N1-Ex.8__ american_users.csv'
INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/tmp/N1-Ex.8__ european_users.csv'
INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/tmp/N1-Ex.8__ companies.csv'
INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;

LOAD DATA INFILE '/tmp/N1-Ex.8__ credit_cards.csv'
INTO TABLE credit_card FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
(id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
```

LOAD DATA INFILE '/tmp/N1-Ex.8__ transactions.csv'
INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;

SET FOREIGN_KEY_CHECKS = 1;

100% 78:189 1 error found

Action	Time	Action	Response	Duration / Fetch Time
1	13:12:05	SET FOREIGN_KEY_CHECKS = 0	0 row(s) affected	0.00026 sec
2	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ american_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS	0 row(s) affected Records: 0 Deleted: 0 Skipped: 0	0.00026 sec
3	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ european_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS	0 row(s) affected Records: 0 Deleted: 0 Skipped: 0	0.0031 sec
4	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ companies.csv' INTO TABLE company FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0	0.0029 sec
5	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0	0.059 sec
6	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ transactions.csv' INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0	1.975 sec
7	13:12:07	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.00022 sec

Query Completed

Local instance 3306 - Warning - not supported MySQL Model* EER Diagram EER Diagram2 EER Diagram3

Schemas

ejercicio8

Tables

Views

Stored Procedures

Functions

sys

tienda_online

transactions

world

```

200
201
202 (id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
203 SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
204
205 • LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... transactions.csv'
206 INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
207
208 • SELECT country, COUNT(*) as total
209 FROM user
210 GROUP BY country;
211
212

```

100% 1/271 1 error found

Result Grid

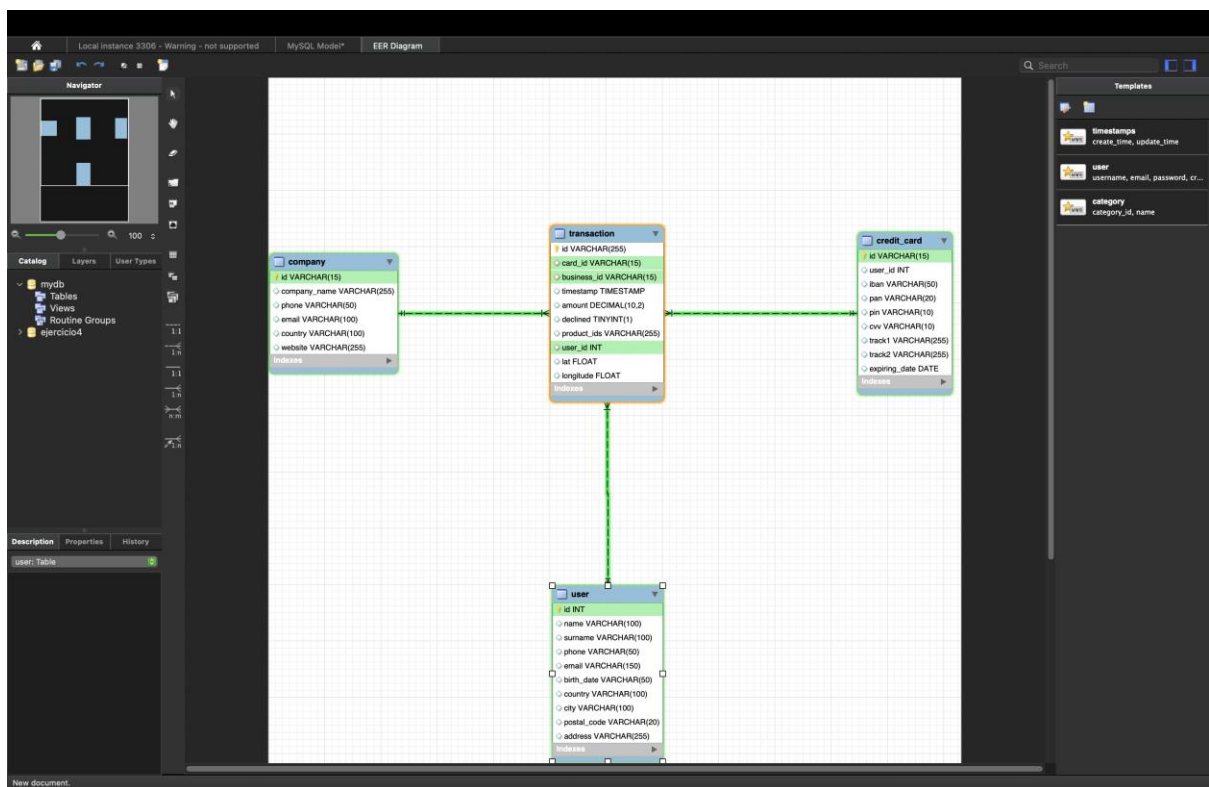
country	total
United States	520
United Kingdom	476
Canada	460
Spain	461
Netherlands	405
Sweden	417
France	432
Poland	415
Germany	453
Italy	439
Portugal	441

Result 17

Action Output

Time	Action	Response	Duration / Fetch Time
18:33:17	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(10), create_time TIMESTAMP)	0 row(s) affected	0.0052 sec
18:33:17	CREATE TABLE company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(100), website VARCHAR(255))	0 row(s) affected	0.0043 sec
18:33:17	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date DATE)	0 row(s) affected	0.0057 sec
18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... american_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	1010 row(s) affected; 1 warning(s): '1681 Integer display...'	0.015 sec
18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... european_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	3990 row(s) affected; 3990 Deleted: 0 Skip...	0.053 sec
18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... companies.csv' INTO TABLE company FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	100 row(s) affected; 100 Deleted: 0 Skip...	0.0026 sec
18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	5000 row(s) affected; 5000 Deleted: 0 Skip...	0.051 sec
18:40:05	LOAD DATA INFILE '/Users/leonardoruggiero/Documents/Curso Especializao 2026 SQL/N1.Ex.8... transactions.csv' INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	100000 row(s) affected; 100000 Deleted: 0 Skip...	2.672 sec
18:41:03	SELECT country, COUNT(*) as total FROM user GROUP BY country;	11 row(s) returned	0.0037 sec / 0.00002...

Query Completed



Respuesta:

En primer lugar, analicé cada archivo CSV y registré sus respectivas columnas para proceder con la **creación de las tablas** user (unificando los archivos de american_users y european_users), company, credit_card y transaction. Diseñé cada estructura utilizando el formato de datos más óptimo y estableciendo **foreign keys** para vincular los registros de manera adecuada. (hice un pantallazo con un select para confirmar si la union de las tablas de users estava correcta). Luego durante este proceso, identifiqué que la columna **expiring_date** no venía en formato de fecha, por lo que fue necesario transformarla utilizando la función **STR_TO_DATE** al momento de cargar la tabla credit_card. Para la integración de los datos, utilicé una línea de código 'INSERT' para cargar los archivos CSV, permitiendo que el SQL los procese e inserte en las tablas creadas.

Ejercicio 9

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
263
264 -- Ejercicio 9
265 -- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.
266
267
268 * SELECT id, name, surname, country
269 FROM user as u
270 WHERE EXISTS (
271     SELECT user_id
272     FROM transaction as t
273     WHERE t.user_id = u.id and declined = 0
274     GROUP BY t.user_id
275     HAVING COUNT(*) > 80);
276
277
278
```

The Results tab shows the following data:

id	name	surname	country
185	Molly	Gilliam	United Kingdom
289	Dwight	Hewitt	Germany
318	Bryar	Adams	Italy

The Action Output tab shows the execution details:

Time	Action	Response	Duration / Fetch Time
34 12:55:30	SELECT id, name, surname, country FROM user as u WHERE EXISTS (SELECT user_id FROM transaction as t WHERE t.user_id = u.id and declined = 0 GROUP BY t.u...	3 row(s) returned	0.272 sec / 0.000022...
35 12:55:20	SELECT id, name, surname, country FROM user as u WHERE EXISTS (SELECT user_id FROM transaction as t WHERE t.user_id = u.id and declined = 0 GROUP BY t.u...	3 row(s) returned	0.279 sec / 0.000019...

RESPUESTA:

Primero hice un "SELECT" del "id" de la tabla de "user" seleccionando también "name", "surname" y "country" para que el resultado sea más visible, relacionando el "id" con la subconsulta que busca el "user_id" de la tabla "transaction", aplicando las funciones "GROUP BY" y "HAVING COUNT (*) > 80". De esta manera, el código realiza el filtro en la subconsulta según lo solicitado y lo compara con el "SELECT" de la consulta principal. He añadido "WHERE declined = 0" para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 10

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
311 HAVING COUNT(*) > 80);
312
313
314 -- Ejercicio 10
315 -- Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.
316
317
318 SELECT AVG(AMOUNT) AS media_amount, company_name, iban
319 FROM transaction
320 JOIN credit_card ON transaction.credit_card_id = credit_card.id
321 JOIN company ON transaction.company_id = company.id
322 WHERE company_name = "Donec Ltd" AND declined = 0
323 GROUP BY iban;
324
325
326 -- Nivel 2
327
328 -- Ejercicio 1
329 -- Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.
330
331 SELECT DATE(timestamp) AS dias, SUM(amount) AS mayor_cantidad_de_ingresos
```

The Results tab shows the following data:

media_amount	company_name	iban
139.580000	Donec Ltd	XX07787529178445952975555640
257.970000	Donec Ltd	XX0413827382289719304958990
159.580000	Donec Ltd	XX0331722440727695610760008
248.410000	Donec Ltd	XX6887684385432090894804602
188.580000	Donec Ltd	MX28358861538898349
439.390000	Donec Ltd	PT764698356665507614343404576
141.560000	Donec Ltd	XX804008331154918341803913
189.210000	Donec Ltd	

The Action Output tab shows the following query execution details:

Time	Action	Response	Duration / Fetch Time
13:53:16	SELECT AVG(AMOUNT) AS media_amount, company_name, iban FROM transaction JOIN credit_card ON transaction.credit_card_id = credit_card.id JOIN company ON tr...	370 row(s) returned	0.010 sec / 0.00012 s...

RESPUESTA:

Empecé la construcción del ejercicio haciendo una selección del 'avg' (promedio) de 'amount', seleccionando el 'iban' y el 'business_id' para filtrar la respuesta. Luego, realicé un 'JOIN' uniendo las tablas ('credit_card' = 'transaction') y ('transaction' = 'company') por las claves primarias para saber cuántos ibans y valores de transacciones fueran realizados por la emplza Donec Ltd usando 'where = Donec Ltd.' y 'where = declined = 0' para eliminar transacciones nulas, finalizando con el 'group by' agrupando por el iban."

Nivel 2

Ejercicio 1

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, showing tables 'company', 'credit_card', 'transaction', and 'user'. The main editor contains a SQL query for 'Ejercicio 1' that identifies the five days with the highest total revenue. The query uses a subquery to find the top 5 days by revenue and then joins this with the 'transaction' table to show the company names. The results are displayed in a table with columns 'dias', 'mayor_cantidad_de_ingre...', and 'company_name'. The bottom panel shows the 'Action Output' with a list of SQL statements and their execution details, including error messages for some queries.

dias	mayor_cantidad_de_ingre...	company_name
2024-12-02	3398.40	Egel Ipsum Ltd
2019-03-19	3149.04	Ac Fermentum Incorporated
2017-12-20	2774.04	Nulla Integer Vulputate Corp.
2017-12-20	2773.35	Viverra Donec Eundem
2020-12-11	2717.76	Ac Fermentum Incorporated

RESPUESTA:

Comencé filtrando los valores solicitados en el enunciado, que son: la fecha (utilicé **DATE(timestamp)**), el **SUM(amount)** para calcular la suma de las transacciones y el nombre de la empresa, que entra en los filtros seleccionados uniendo la tabla

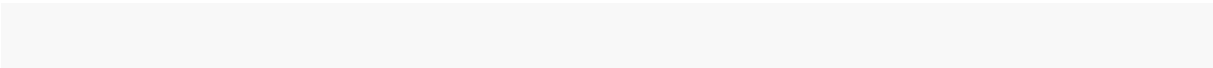
transaction con **company** para obtener el nombre de las empresas. Tras esta construcción, solo faltaron las funciones:

Transacciones aprobadas (declined = 0).

Agrupacion por días y nombre de las empresas (GROUP BY días, company_name).

Las cinco primeras del tope (ORDER BY mayor_cantidad_de_ingresos DESC, LIMIT 5).

Y así obtener el resultado pedido en el ejercicio.



Ejercicio 2

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
307 -- Ejercicio 2
308 -- Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido
309 -- entre 350 y 400 euros y en alguna de estas fechas:
310 -- 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.
311
312 SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined
313 FROM transaction as t
314 JOIN company as c
315 ON t.business_id = c.id
316 WHERE t.amount BETWEEN 350 AND 400
317 AND (DATE(t.timestamp) = "2015-04-29" or DATE(t.timestamp) = "2018-07-20" or DATE(t.timestamp) = "2024-03-13")
318 AND t.declined = "0"
319 ORDER BY amount DESC;
```

The Results grid shows the following data:

company_name	phone	country	fecha	amount	declined
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	399.84	0
Auctor Mauris Vel LLP	08 59 28 74 14	United States	2018-07-20	386.51	0
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	386.29	0
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	386.29	0
Oni Adipiscing Limited	03 15 00 77 81	United Kingdom	2015-04-29	372.17	0
Fringilla LLC	08 29 15 63 57	New Zealand	2015-04-29	367.62	0
Pede Cum Ltd	07 62 28 48 38	Norway	2018-07-20	356.87	0
Auctor Mauris Vel LLP	08 59 28 74 14	United States	2024-03-13	353.75	0

The Action Output pane shows the execution details:

Time	Action	Response	Duration / Fetch Time
59 13:36:33	SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined FROM transaction as t JOIN company as c ON t.business_id = c.id WHERE t.amount...	8 row(s) returned	0.051 sec / 0.000012...
60 13:36:39	SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined FROM transaction as t JOIN company as c ON t.business_id = c.id WHERE t.amount...	8 row(s) returned	0.051 sec / 0.000014...

RESPUESTA

"En este ejercicio, comencé identificando las tablas necesarias para conectar la información:

-**'transaction'** para los montos y fechas

- **'company'** para los nombres y datos de contacto.

-Realicé un 'JOIN' uniendo ambas tablas a través de la clave **'business_id'** de la TRANSACTION y el **'company_id'** de la COMPANY para poder visualizar el nombre de la compañía junto a sus ventas.

-Posteriormente, construí el filtro principal en la cláusula 'WHERE' utilizando el operador **'BETWEEN'** para delimitar los valores de 'AMOUNT' exclusivamente entre **350 y 400 euros**. Para la selección de las fechas, utilicé la función **'DATE()'** sobre el campo 'TIMESTAMP' para ignorar las horas y apliqué la condicional **'WHERE'**, lo que me permitió hacer la condicion con 'OR' para agregar la condicion del enunciado (FECHA = **29 de abril de 2015 or 20 de julio de 2018 or 13 de marzo de 2024**) en una sola condición lógica y agrego la condicion **'declined = 0'** evitando transacciones rechazadas.

-Finalmente, seleccioné las columnas solicitadas (company_name, phone, country, fecha y amount) y terminé la construcción aplicando un 'ORDER BY' sobre el monto de manera descendente, organizando así los resultados de mayor a menor cantidad.

Ejercicio 3

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, showing tables like 'company', 'transaction', and 'credit_card'. The main editor contains a SQL query for 'Ejercicio 3' that filters transactions by amount (between 350 and 400) and date (2015-04-29, 2018-06-20, 2024-03-13), and orders them by amount descending. The 'Result Grid' shows the output with columns: company_name, total_transaction, and conteaje_valores. The results list companies like 'Lorem Eu Incorporated' and 'Fringilla LLC' with their respective transaction counts and values.

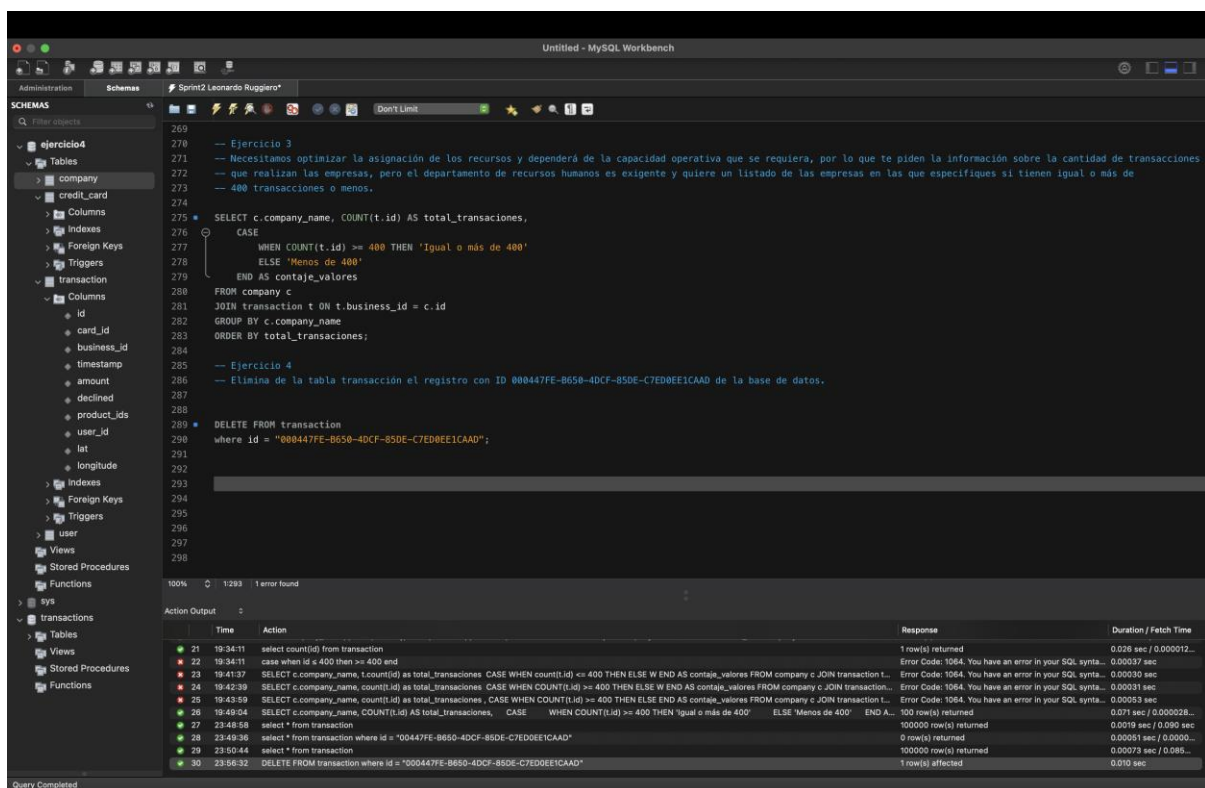
company_name	total_transaction	conteaje_valores
Lorem Eu Incorporated	380	Menos de 400
Fringilla LLC	397	Menos de 400
Nec Luctus LLC	398	Menos de 400
Qui Quis Institute	402	Igual o más de 400
Riis Associates	407	Igual o más de 400
Magna A Neque Industries	410	Igual o más de 400

The 'Action Output' pane at the bottom shows the execution of the query, including the SQL statement and the response from the database.

RESPUESTA:

Para cumplir con lo solicitado, yo hice una consulta vinculando **company** y **transaction** mediante un “**JOIN**”. Utilice “**COUNT**” y “**GROUP BY**” para totalizar las ventas por empresa. La lógica clave se con “**CASE WHEN**”, definiendo que si el conteo es igual o mayor a 400, se asigne el texto '**Igual o más de 400**', y mediante “**ELSE**”, se etiquete el resto como '**Menos de 400**'. El uso de comillas simples fue esencial para identificar los textos, logrando un reporte que categoriza la capacidad operativa para Recursos Humanos.

Ejercicio 4



The screenshot shows the MySQL Workbench interface with a SQL editor and an Action Output window. The SQL editor contains the following queries:

```
269 -- Ejercicio 3
270 -- Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones
271 -- que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de
272 -- 400 transacciones o menos.
273
274
275 SELECT c.company_name, COUNT(t.id) AS total_transacciones,
276 CASE
277 WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400'
278 ELSE 'Menos de 400'
279 END AS contaje_valores
280 FROM company c
281 JOIN transaction t ON t.business_id = c.id
282 GROUP BY c.company_name
283 ORDER BY total_transacciones;
284
285 -- Ejercicio 4
286 -- Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
287
288 DELETE FROM transaction
289 where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
290
291
292
293
294
295
296
297
298
```

The Action Output window shows the execution results of the queries:

Time	Action	Response	Duration / Fetch Time
21 19:34:11	select count(id) from transaction	1 row(s) returned	0.026 sec / 0.00012...
22 19:34:11	case when id <= 400 then >= 400 end	Error Code: 1064. You have an error in your SQL synta...	0.00037 sec
23 19:41:37	SELECT c.company_name, t.count(id) as total_transacciones CASE WHEN count(t.id) <= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction t...	Error Code: 1064. You have an error in your SQL synta...	0.00030 sec
24 19:42:39	SELECT c.company_name, count(t.id) as total_transacciones CASE WHEN COUNT(t.id) >= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction...	Error Code: 1064. You have an error in your SQL synta...	0.00031 sec
25 19:43:59	SELECT c.company_name, count(t.id) as total_transacciones CASE WHEN COUNT(t.id) >= 400 THEN ELSE AS contaje_valores FROM company c JOIN transaction L...	Error Code: 1064. You have an error in your SQL synta...	0.00053 sec
26 19:49:04	SELECT c.company_name, COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400' ELSE 'Menos de 400' END A...	100 row(s) returned	0.071 sec / 0.000028...
27 23:48:58	select * from transaction	100000 row(s) returned	0.0019 sec / 0.090 sec
28 23:49:36	select * from transaction where id = "00447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	0 row(s) returned	0.00051 sec / 0.0000...
29 23:50:44	select * from transaction	100000 row(s) returned	0.00073 sec / 0.086...
30 23:56:32	DELETE FROM transaction where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected	0.010 sec

RESPUESTA:

En este ejercicio, utilicé la instrucción 'DELETE FROM transaction' junto con la cláusula 'WHERE' para localizar el ID solicitado '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' y realizar la eliminación requerida en el enunciado.

The screenshot displays a MySQL IDE interface with a SQL script in the main editor and its execution results in the bottom pane.

SQL Script:

```

277 WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400'
278 ELSE 'Menos de 400'
279 END AS contaje_valores
280 FROM company c
281 JOIN transaction t ON t.business_id = c.id
282 GROUP BY c.company_name
283 ORDER BY total_transacciones;
284
285 -- Ejercicio 4
286 -- Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
287
288
289 DELETE FROM transaction
290 where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
291
292
293 SELECT * FROM transaction
294 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
295
296
297
298

```

Execution Results:

Time	Action	Response	Duration / Fetch Time
23 19:41:37	SELECT c.company_name, t.count(id) as total_transacciones, CASE WHEN count(t.id) <= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction t...	Error Code: 1064. You have an error in your SQL synta...	0.00030 sec
24 19:42:39	SELECT c.company_name, count(t.id) as total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction...	Error Code: 1064. You have an error in your SQL synta...	0.00031 sec
25 19:43:59	SELECT c.company_name, count(t.id) as total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN ELSE END AS contaje_valores FROM company c JOIN transaction...	Error Code: 1064. You have an error in your SQL synta...	0.00053 sec
26 19:49:04	SELECT c.company_name, COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400' ELSE 'Menos de 400' END A...	100 row(s) returned	0.071 sec / 0.000030...
27 23:48:58	select * from transaction	100000 row(s) returned	0.0019 sec / 0.000...
28 23:49:36	select * from transaction where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	0 row(s) returned	0.00051 sec / 0.0000...
29 23:50:44	select * from transaction	100000 row(s) returned	0.00073 sec / 0.008...
30 23:56:32	DELETE FROM transaction where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected	0.010 sec
31 23:58:00	SELECT * FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	Error Code: 1146. Table 'ejercicio4.transactions' does...	0.0011 sec
32 23:58:29	SELECT * FROM transaction WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	0 row(s) returned	0.00064 sec / 0.000...

RESPUESTA:

También realicé una captura de pantalla (printscreen) de una consulta 'SELECT' filtrando por ese mismo ID para confirmar que todo fue eliminado correctamente y que el registro ya no existe en la base de datos.

Ejercicio 5

Local instance 3306 - Warning - not supported | MySQL Model* | EER Diagram

Schemas

ejercicio4

- company
- credit_card
- transaction
- user
- Views
 - vistamarketing
- Stored Procedures
- Functions
- sys
- transactions

293 SELECT * FROM transaction

294 WHERE id = '000447FE-B650-4DCF-850E-C7ED0EE1CA0D';

295

296

297 -- Ejercicio 5

298 -- La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione

299 -- detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía.

300 -- Teléfono de contacto, País de residencia, Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

301

302 CREATE VIEW VistaMarketing AS

303 SELECT company_name, phone, country, AVG(amount) AS media_por_compania

304 FROM company

305 JOIN transaction ON company.id = transaction.business_id

306 WHERE declined = 0

307 group by company_name, phone, country;

308

309 SELECT * FROM VistaMarketing

310 ORDER BY media_por_compania DESC;

311

312

313

314

100% 1:31 1 error found

Result Grid

company_name	phone	country	media_por_compa...
Ac Fermentum Incorporated	06 85 58 52 33	Germany	284.911333
Perum Nique Corp.	07 77 48 85 28	Australia	275.876041
Uma Convalis Associates	06 01 24 77 04	United States	272.568925
Al Associates	09 58 61 10 85	New Zealand	272.736985
Matta Viter Associates	08 28 44 40 58	Australia	270.251876
Alquist Diam Limited	02 78 61 47 48	United States	269.291145

VistaMarketing 50

Action Output

Time	Action	Response	Duration / Fetch Time
34 10:23:41	SELECT company_name, company_id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON company.id = transaction.business_id...	Error Code: 1140. In aggregated query without GROU...	0.0024 sec
35 10:32:12	CREATE VIEW VistaMarketing AS SELECT company_name, company_id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	0 row(s) affected	0.0081 sec
36 10:32:12	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	Error Code: 1055. Expression #2 of SELECT list is not...	0.0012 sec
37 10:32:33	CREATE VIEW VistaMarketing AS SELECT company_name, company_id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	Error Code: 1050. Table 'VistaMarketing' already exists	0.0012 sec
38 10:33:27	DROP VIEW VistaMarketing	0 row(s) affected	0.0027 sec
39 10:34:04	CREATE VIEW VistaMarketing AS SELECT company_name, company_id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	0 row(s) affected	0.0024 sec
40 10:34:42	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	Error Code: 1055. Expression #2 of SELECT list is not...	0.0012 sec
41 10:35:18	DROP VIEW VistaMarketing	0 row(s) affected	0.0018 sec
42 10:35:28	CREATE VIEW VistaMarketing AS SELECT company_name, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON company.id = tran...	0 row(s) affected	0.0023 sec
43 10:35:34	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	100 row(s) returned	0.272 sec / 0.000035...

Query Completed

RESPUESTA:

Primero construí la vista y la nombré como pide el ejercicio usando "**CREATE VIEW as (nombre de la vista)**", luego creé los "**SELECT**" junto con la media de las transacciones que se pide al principio del ejercicio. Realicé un "**JOIN**" uniendo la tabla "**company**" con la tabla "**transaction**" porque el "**amount**" del cual necesito obtener la media está en la tabla de transacción. Usé la condición "**WHERE declined = 0**" para devolver solo transacciones efectuadas y agrupé con "**GROUP BY**" por las columnas "**company_name**", "**phone**" y "**country**". Al final, hice el "**SELECT**" de toda la vista ordenando la media de mayor a menor con "**ORDER BY DESC**" para filtrar el resultado deseado.

Nivel 3

Ejercicio 1

The screenshot shows the MySQL Workbench interface. The top toolbar includes icons for file operations, connection, and execution. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected. The main editor contains a SQL query for 'Nivel 3 Ejercicio 1'. The query creates a view 'VistaMarketing' and then selects data from it, using window functions to determine card status based on the last three transactions. The 'Result Grid' at the bottom shows the output of the query, with columns 'status' and 'total_tarjetas'. The 'Action Output' pane at the very bottom shows the execution log with timestamps and messages for each step of the query execution.

```
WITH UltimasTransacciones AS
(SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as fila
FROM transaction)

322 SELECT status, COUNT(*) AS total_tarjetas
323 FROM (SELECT card_id,
324 CASE WHEN SUM(declined) = COUNT(declined) AND COUNT(declined) >= 3 THEN 'Inactive'
325 ELSE 'Activo'
326 END AS status
327 FROM UltimasTransacciones
328 WHERE fila <= 3
329 GROUP BY card_id
330 ) AS ResumenEstado
331 GROUP BY status
332
```

status	total_tarjetas
Activo	4995
Inactivo	5

RESPUESTA:

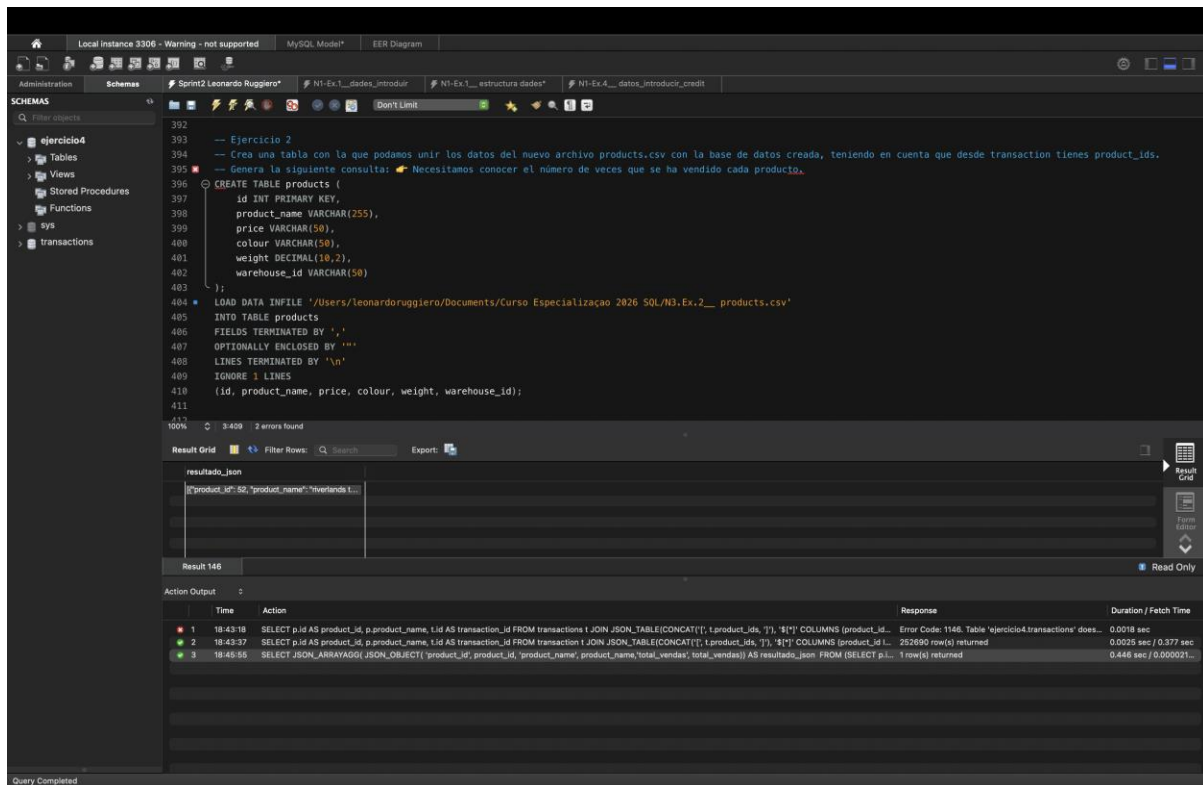
En este ejercicio, utilicé **Funciones de Ventana (Window Functions)** para evaluar el comportamiento reciente de cada tarjeta. El proceso se detalla a continuación:

Primero, creé una **Expresión de Tabla Común** llamada 'UltimasTransacciones'. Dentro de ella, utilicé la función '**ROW_NUMBER()**' combinada con '**OVER(PARTITION BY card_id ORDER BY timestamp DESC)**'. Esta instrucción es fundamental porque 'numera' las transacciones de cada tarjeta individualmente, asignando el número 1 a la más reciente y continuando en orden descendente.

Después, en la consulta principal, filtré los resultados para considerar únicamente las filas donde el número asignado fuera **menor o igual a 3**. Sobre este subconjunto de datos, apliqué una lógica de agregación: si la suma de las transacciones marcadas como rechazadas ('declined' = 1) es igual al total de transacciones analizadas para esa tarjeta, el sistema le asigna el estado de '**Inactivo**'. En cualquier otro caso (si hay al menos una transacción exitosa), la tarjeta se clasifica como '**Activo**'.

Finalmente, agrupé los resultados por el nuevo estado generado para obtener el conteo total. El análisis confirmó que existen **4.995 tarjetas activas**, ya que la gran mayoría presentó al menos un movimiento exitoso dentro de sus últimas tres operaciones

Ejercicio 2



RESPUESTA:

Primero usé el comando **CREATE TABLE** para fabricar el "molde" o la caja donde guardaría los productos. Definí qué columnas necesitaba (ID, nombre, precio, etc.) y le puse una **Primary Key** al ID. Hice esto para que cada producto sea único y no se mezclen los datos. Con la tabla ya creada, usé el comando **INSERT INTO** para meter la información que venía en el archivo CSV, simplemente "llené" los huecos. Me aseguré de que cada dato cayera en su columna correspondiente (el nombre en la columna de nombres, el precio en la de precios). Es el paso donde la tabla cobra vida con datos reales.

Resumen de las funciones que apliqué:

CONCAT(): La usé para "engañar" al sistema y que viera la columna como una lista JSON válida.

JSON_TABLE(): Mi herramienta clave para convertir esa lista en filas de una tabla.

JSON_OBJECT(): La usé para darle nombre a las etiquetas dentro del resultado (como "total_vendas").

JSON_ARRAYAGG(): Con esta agrupé todas las filas en un solo array final.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
412
413
414 SELECT p.id AS product_id, p.product_name, t.id AS transaction_id
415 FROM transaction t
416 JOIN JSON_TABLE(CONCAT('[', t.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')) AS jt ON TRUE
417 JOIN products p ON p.id = jt.product_id
418 WHERE t.declined = 0;
419
420
421 SELECT JSON_ARRAYAGG( JSON_OBJECT( 'product_id', product_id, 'product_name', product_name, 'total_vendas', total_vendas)) AS resultado_json
422
423 FROM (SELECT p.id AS product_id, p.product_name, COUNT(*) AS total_vendas
424 FROM transaction t
425 JOIN JSON_TABLE( CONCAT('[', t.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')) AS jt ON TRUE
426 JOIN products p ON p.id = jt.product_id
427 WHERE t.declined = 0
428 GROUP BY p.id, p.product_name
429 ORDER BY total_vendas DESC
430 ) AS summary;
431
```

The Results tab shows a table with the following data:

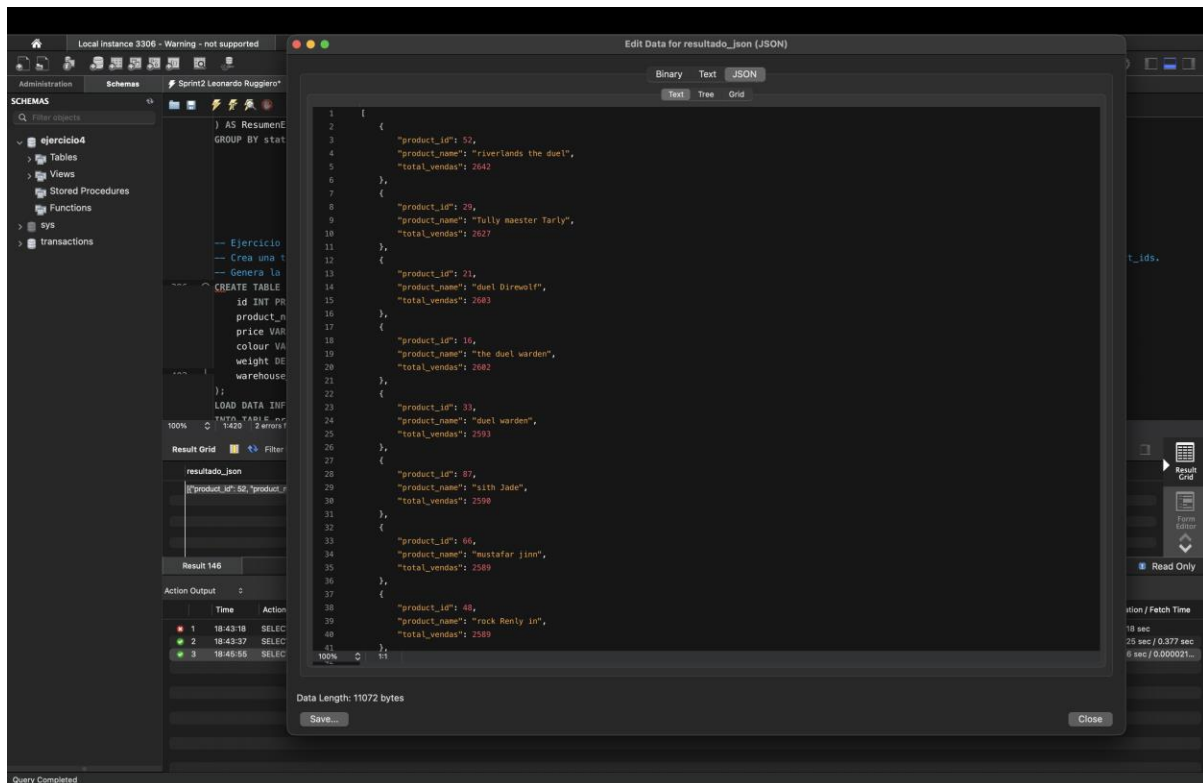
product_id	product_name	total_vendas
80	Overland	1

The Action Output tab shows the following actions:

Time	Action
18:43:18	SELECT p.id AS product_id
18:43:37	SELECT p.id AS product_id
18:45:55	SELECT JSON_ARRAYAGG(JSON_OBJECT('product_id', product_id, 'product_name', product_name, 'total_vendas', total_vendas)) AS resultado_json

The Response tab shows the following error message:

```
Error Code: 1146, Table 'ejercicio4.transactions' does not exist
transaction t JOIN JSON_TABLE(CONCAT('[', t.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')) AS jt ON TRUE
252690 row(s) returned
0.0025 sec / 0.377 sec
1 row(s) returned
0.446 sec / 0.000021...
```



Una vez que terminé de ejecutar todo el código, utilicé el **Viewer** de MySQL Workbench para consultar el resultado final, ya que ahí es donde quedan guardados y organizados los datos de la consulta de una forma legible. Para hacerlo, seguí estos pasos:

En la cuadrícula de resultados, busqué la celda que contenía el **JSON** generado.

Hice **clic derecho** sobre esa celda y seleccioné la opción **"Open Value in Viewer"**.

Dentro del visor, me aseguré de marcar la pestaña **"JSON"**.

Hice esto porque, al ser un reporte complejo, el visor me permite expandir y contraer los datos para validar que cada producto tuviera su conteo de ventas correcto antes de dar el ejercicio por finalizado.